

Metodologie di Programmazione (A-L)

II semestre - a.a. 2025 – 2026

Strutture dati II

a cura di Massimo La Morgia



SAPIENZA
UNIVERSITÀ DI ROMA

*Tutti i diritti relativi al presente materiale didattico ed al suo contenuto sono riservati a Sapienza e ai suoi autori (o docenti che lo hanno prodotto). È consentito l'uso personale dello stesso da parte dello studente a fini di studio. Ne è vietata nel modo più assoluto la diffusione, duplicazione, cessione, trasmissione, distribuzione a terzi o al pubblico pena le sanzioni applicabili per legge.

**I crediti sulle slide di questo corso sono riportati nell'ultima slide

Strutture Dati

parte II

Algoritmi sulle collezioni: la classe `java.util.Collections`

- Fornisce metodi **statici** per la **manipolazione delle collezioni**:

Metodo	Descrizione
sort	Ordina gli elementi di una List
binarySearch	Cerca un elemento di una List mediante ricerca binaria
fill	Rimpiazza tutti gli elementi di una List con l'elemento specificato
copy	Copia gli elementi da una lista all'altra
reverse	Inverte l'ordine degli elementi di una List
shuffle	Mette in ordine casuale gli elementi di una List
min/max	Restituisce l'elemento più piccolo/grande della Collection

Algoritmi sugli array: la classe `java.util.Arrays`

- Fornisce metodi `statici` per la `manipolazione` degli array:

Metodo	Descrizione
<code>sort</code>	Ordina gli elementi dell'array
<code>binarySearch</code>	Cerca un elemento mediante ricerca binaria
<code>fill</code>	Riempì l'array con l'elemento specificato
<code>copyOf</code>	Restituisce una copia di un array
<code>equals</code>	Confronta due array elemento per elemento
<code>asList</code>	Restituisce una lista contenente gli elementi dell'array
<code>toString</code>	Restituisce una rappresentazione dell'array sotto forma di stringa

Ordinamento "naturale"

- Come **garantire un ordinamento sui tipi** utilizzati nelle strutture dati che si fondano su un ordinamento (es. TreeSet o TreeMap)?
- E' necessario che quei tipi implementino un'interfaccia speciale, chiamata **Comparable<T>**
- Dotata di un solo metodo:

Metodo	Descrizione
int compareTo (T o)	Confronta sé stesso con l'oggetto o (restituendo 0 se uguali, -1 se <o, +1 altrimenti)

L'interfaccia Comparable: un esempio

- Come garantire l'ordinamento di una classe NomeCognome?

```
public class NomeCognome implements Comparable<NomeCognome>
{
    private String nome;
    private String cognome;

    public NomeCognome(String nome, String cognome)
    {
        this.nome = nome;
        this.cognome = cognome;
    }

    @Override
    public int compareTo(NomeCognome o)
    {
        int v = cognome.compareTo(o.cognome);

        if (v == 0) return nome.compareTo(o.nome);
        else return v;
    }

    @Override
    public String toString() { return cognome+" "+nome; }
}
```

Ordinamento con l'interfaccia `Comparator<T>`

- E se la classe **non fornisce** "di suo" **l'ordinamento** mediante l'interfaccia `Comparable`?
- E se voglio **ordinare diversamente** gli elementi di un certo oggetto?
- E' sufficiente implementare un'interfaccia `Comparator<T>` e passarne un'istanza in input al costruttore delle strutture dati (es. `TreeSet` e `TreeMap`)
- Metodi astratti dell'interfaccia `Comparator<T>`

Method Summary	
int	<code>compare</code> (<code>T o1, T o2)</code>
	Compares its two arguments for order.

Riferimenti a metodi esistenti

- Se una lambda si limita a chiamare un metodo, allora si possono usare i **riferimenti a metodi**
- Utilizzando la sintassi:

```
NomeClasse::metodoIstanza // Il primo parametro è l'oggetto su cui è invocato
                           // il metodo e gli eventuali altri parametri sono
                           // passati al metodo

NomeClasse::metodoStatico // Tutti i parametri sono passati al metodo

oggetto::metodoIstanza    // Tutti i parametri sono passati al metodo

NomeClasse::new           // Tutti i parametri sono passati al metodo
```

Riferimenti a metodi esistenti

- Esempio:

Interfaccia

```
@FunctionalInterface
public interface Converter<F, T>
{
    T convert(F from);
}
```

Implementazione con classe anonima

```
Converter<String, Integer> converter = new Converter<String, Integer>() {
    @Override
    public Integer convert(String from) {
        return Integer.valueOf(from);
    }
};
```

Implementazione con lambda function

```
Converter<String, Integer> converter = a->Integer.valueOf(a);
```

Implementazione con method reference

```
Converter<String, Integer> converter = Integer::valueOf;
```

Confronto tra riferimenti ed espressioni lambda

Riferimento a metodo	Tipo di metodo riferito	Espressione lambda equivalente
String::valueOf	Statico	x -> String.valueOf(x)
Object::toString	D'istanza, senza fissare l'istanza (intestazione estesa)	x -> x.toString()
x::toString	D'istanza, fissata l'istanza	() -> x.toString()
ArrayList::new	Costruttore	() -> new ArrayList<>()

Riferimento a metodi d'istanza usando il nome della classe vs. usando un riferimento a un oggetto

- Che differenza c'è tra:
 - `riferimentoOggetto::metodoNONStatico` e
 - `nomeClasse::metodoNONStatico`?
- Nel primo caso, il metodo sarà chiamato **sull'oggetto riferito**
- Nel secondo caso, **non stiamo specificando su quale oggetto applicare il metodo**
 - Ma il metodo è d'istanza, quindi utilizza membri d'istanza (campi, metodi, ecc.)
 - Ci si riferisce al metodo **implicitamente sovraccaricato (dal compilatore)** con un primo parametro aggiuntivo: un riferimento a un oggetto della classe cui appartiene il metodo



Esempio di riferimento a metodi d'istanza mediante classe

- Considerate `Arrays.sort(T[] a, Comparator<? super T> c)`
- E l'interfaccia `Comparator<T>`:

@FunctionalInterface

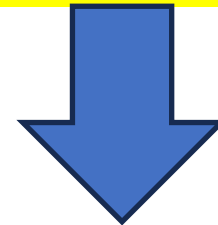
```
public interface Comparator<T>
{
    int compare(T o1, T o2);
}
```

Il metodo d'istanza `int compareTo(String o) {...}` della classe `String` viene implicitamente sovraccaricato con un nuovo metodo: 4

```
int compareTo(String riferimento, String o){
    return riferimento.compareTo(o);
}
```

- Posso scrivere:

```
Arrays.sort(new String[] { "a", "c", "b" }, String::compareTo);
```



Visibilità dalle espressioni lambda

- Accesso alla variabili esterne da un'espressione lambda simile a quello di una classe anonima
- Si possono accedere:
 - Campi d'istanza e variabili statiche
 - Variabili **final** del metodo che definisce la lambda
 - Variabili del metodo esterno **implicitamente final**
- Esempio:

```
int num = 1; // non viene modificata all'interno del metodo  
Converter<Integer, String> stringConverter =  
    from -> String.valueOf(from + num);  
stringConverter.convert(2); // "3"
```



Ordinamento inverso: da Java 7 a Java 8

```
List<String> names = Arrays.asList("peter", "anna", "mike", "xenia");
```

In Java 7:

```
Collections.sort(names, new Comparator<String>()  
{  
    @Override  
    public int compare(String a, String b) { return b.compareTo(a); }  
});
```

In Java 8:

```
Collections.sort(names, (a, b) -> b.compareTo(a));
```



Ordinamento per lunghezza: da Java 7 a Java 8

```
List<String> names = Arrays.asList("peter", "anna", "mike", "xenia");
```

In Java 7:

```
Collections.sort(names, new Comparator<String>()  
{  
    @Override  
    public int compare(String a, String b)  
    {  
        return Integer.compare(a.length(), b.length());  
    }  
});
```

In Java 8:

```
Collections.sort(names, (a, b) -> a.length()-b.length());  
    // oppure: (a, b) -> Integer.compare(a.length(), b.length());
```



Esempio

```
class Person
{
    private String firstName;
    private String lastName;
    public Person(String firstName, String lastName)
    {
        this.firstName = firstName;
        this.lastName = lastName;
    }
}
```

Metodi di default nell'interfaccia Comparator

```
Comparator<Person> comparator = (p1, p2) -> p1.firstName.compareTo(p2.firstName);  
Person p1 = new Person("John", "Doe");  
Person p2 = new Person("Alice", "Wonderland");  
comparator.compare(p1, p2); // > 0
```

- Confronto **inverso**:

```
comparator.reversed().compare(p1, p2); // <0
```

- Confronto **per una data chiave** specificata:

```
comparator = Comparator.comparing(p -> p.getFirstName());
```

- Confronto per **criteri multipli a cascata**:

```
comparator = Comparator.comparing(p -> p.getFirstName())  
    .thenComparing(p -> p.getLastName())
```

Metodi di default nell'interfaccia Comparator

- Confronto per criteri multipli a cascata:

```
comparator = Comparator.comparing(p -> p.getFirstName())  
    .thenComparing(p -> p.getLastName())
```

- Confronto per criteri multipli a cascata:

```
comparator = Comparator.comparing(Person::getFirstName)  
    .thenComparing(Person::getLastName)
```

Interfacce funzionali built-in (java.util.function.*)

- **Predicate<T>**: funzione a valori booleani a un solo argomento generico T

```
Predicate<String> predicate = s -> s.length() > 0;  
predicate.test("foo"); // true
```

```
predicate.negate().test("foo"); // false
```

```
Predicate<String> predicate2 = s -> s.startsWith("f");  
predicate.and(predicate2).test("foo"); // true
```

```
Predicate<Boolean> nonNull = Objects::nonNull;
```

```
Predicate<Boolean> isNull = Objects::isNull;
```

```
Predicate<String> isEmpty = String::isEmpty;
```

```
Predicate<String> isEmpty = isEmpty.negate();
```

Interfacce funzionali built-in (java.util.function.*)

- **Predicate<T>**: funzione a valori booleani a un solo argomento generico T

All Methods	Static Methods	Instance Methods	Abstract Methods	Default Methods
Modifier and Type		Method and Description		
default	Predicate<T>	and(Predicate<? super T> other) Returns a composed predicate that represents a short-circuiting logical AND of this predicate and another.		
static	<T> Predicate<T>	isEqual(Object targetRef) Returns a predicate that tests if two arguments are equal according to Objects.equals(Object, Object) .		
default	Predicate<T>	negate() Returns a predicate that represents the logical negation of this predicate.		
default	Predicate<T>	or(Predicate<? super T> other) Returns a composed predicate that represents a short-circuiting logical OR of this predicate and another.		
boolean		test(T t) Evaluates this predicate on the given argument.		

Interfacce funzionali built-in (java.util.function.*)

- **Function<T,R>**: funzione a un argomento di tipo T e un tipo di ritorno R entrambi generici

```
Function<String, Integer> toInteger = Integer::valueOf;
```

```
Integer k = toInteger.apply("123");
```

```
Function<Integer, Integer> square = k -> k*k;
```

```
Integer sqr = square.apply(5); // 25
```

Interfacce funzionali built-in (java.util.function.*)

- **Function<T,R>**: funzione a un argomento di tipo T e un tipo di ritorno R entrambi generici

All Methods

Static Methods

Instance Methods

Abstract Methods

Default Methods

Modifier and Type

Method and Description

default <V> Function<T,V>	andThen(Function<? super R,? extends V> after) Returns a composed function that first applies this function to its input, and then applies the after function to the result.
R	apply(T t) Applies this function to the given argument.
default <V> Function<V,R>	compose(Function<? super V,? extends T> before) Returns a composed function that first applies the before function to its input, and then applies this function to the result.
static <T> Function<T,T>	identity() Returns a function that always returns its input argument.

Interfacce funzionali built-in (java.util.function.*)

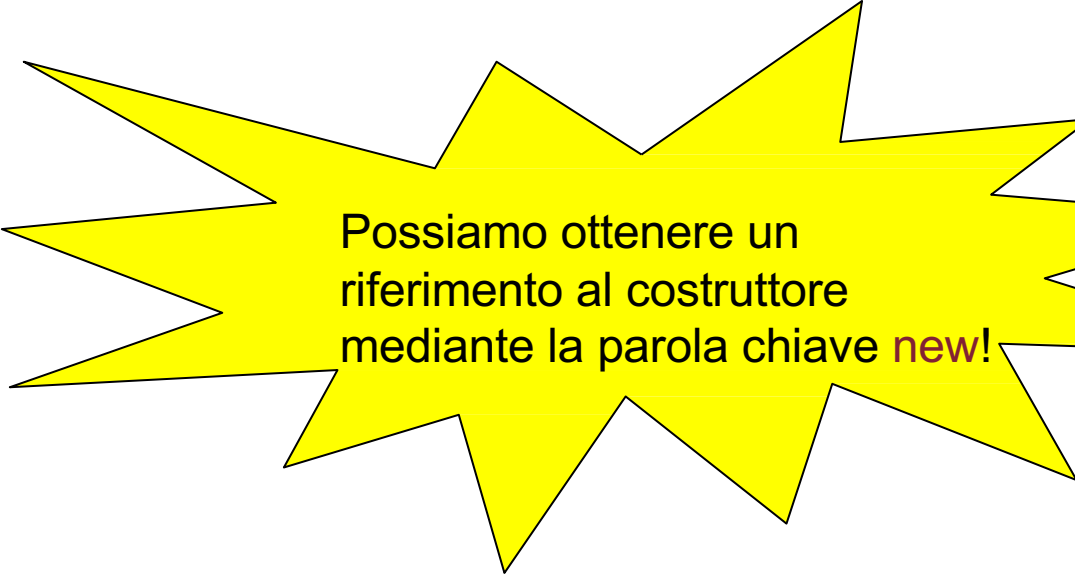
- **Supplier<T>**: funzione senza argomenti in input

```
Supplier<String> stringSupplier = () -> "ciao";
```

Interfacce funzionali built-in (java.util.function.*)

- `Supplier<T>`: funzione senza argomenti in input

```
Supplier<Person> personSupplier = Person::new;  
personSupplier.get(); // new Person()
```



Possiamo ottenere un riferimento al costruttore mediante la parola chiave `new`!

Interfacce funzionali built-in (java.util.function.*)

- **Consumer<T>**: funzione con un argomento di tipo generico T e nessun tipo di ritorno

```
Consumer<Person> greeter1 = p ->  
System.out.println("Hello, " + p.firstName);  
greeter1.accept(new Person("Luke", "Skywalker"));  
Consumer<Person> greeter2 = System.out::println;
```

For each su java.util.Collection

- Le collection sono dotate di un metodo `forEach` che prende in input un'interfaccia `Consumer<? super T>` dove `T` è il tipo generico della collection
- Ad esempio:

```
Collection<String> c = Arrays.asList("aa", "bb", "cc");
```

```
// In Java 7:
```

```
for (String s : c) System.out.println(s);
```

```
// In Java 8:
```

```
c.forEach(s -> System.out.println(s));
```

```
// persino meglio:
```

```
c.forEach(System.out::println);
```

Pila e coda

- Due strutture dati fondamentali utili in un gran numero di attività
- Coda (**FIFO**: First In, First Out)



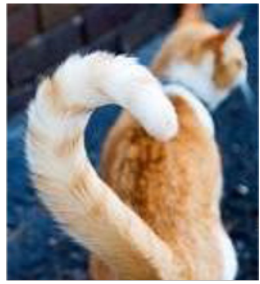
Coda
FIFO

- Pila o stack (**LIFO**: Last In, First Out)



Stack
LIFO

Coda



- Esempi di coda:
 - Coda degli eventi relativi a mouse e tastiera
 - Coda di stampa
- Esistono implementazioni standard della coda mediante l'interfaccia **Queue**
 - **LinkedList** implementa l'interfaccia **Queue**!
- Operazioni principali:
 - **add**: inserisce un elemento in coda
 - **remove**: rimuove un elemento dall'inizio della coda
 - **peek**: restituisce l'elemento all'inizio della coda senza rimuoverlo

Pila



- Esempi di pila:
 - La **pila di esecuzione** (stack) contenente i record di attivazione delle chiamate a metodi
 - Nell'implementazione della **ricorsione...**
- Esiste un'implementazione standard mediante la classe **Stack**
 - Implementa l'interfaccia **List**
- Operazioni principali:
 - **push**: inserisce un elemento in cima alla pila
 - **pop**: rimuove l'elemento in cima alla pila
 - **peek**: restituisce l'elemento in cima alla pila senza rimuoverlo



Esercizio: parentesi annidate

- Scrivere un metodo che, data una stringa in input, verifichi se le parentesi della stringa sono correttamente annidate
 - Sono ammessi due tipi di parentesi: tonde e quadre

```
import java.util.Stack;

public class Parentesi
{
    public boolean annidate(String s)
    {
        Stack<Character> p = new Stack<Character>();

        for (int k = 0; k < s.length(); k++)
        {
            char c = s.charAt(k);
            switch(c)
            {
                case '(': case '[': p.push(c); break;
                case ')': if (p.isEmpty() || p.pop() != '(') return false; break;
                case ']': if (p.isEmpty() || p.pop() != '[') return false; break;
            }
        }

        return p.isEmpty();
    }
}
```



Esercizio: inversione degli elementi

- Scrivere un metodo che, data una coda di interi in input, ne inverta gli elementi
 - Scrivere la variante in cui viene fornita in input (e restituita in output) una stringa

```
public void inverti(Queue<Integer> q)
{
    Stack<Integer> s = new Stack<Integer>();

    while(!q.isEmpty())
    {
        Integer e = q.remove();
        s.push(e);
    }

    while(!s.isEmpty()) q.add(s.pop());
}
```

Scegliere una collection

- Avete tutte queste strutture dati e **dovete scegliere in quale struttura memorizzare i vostri oggetti**

– Che cosa fate?

1. stracciate tutto



2. capite **a cosa serve** ciascuna collezione!

Alcune domande utili (1)

- Come voglio **accedere** agli elementi?
 - Tramite una **posizione**? ArrayList...
 - Tramite una **chiave**? una mappa...
 - **Non importa**: un insieme...
- Quali sono i **tipi degli elementi** o **tipi di chiavi e valori**?
 - Ad esempio, in una mappa, vado da stringhe a interi o da interi a stringhe? Qual è la chiave univoca che deve determinare il valore?
- L'**ordine degli elementi** o delle chiavi è importante?
 - Sì, in un **ordine naturale** degli elementi: TreeMap o TreeSet
 - Sì, nell'**ordine di inserimento**: ArrayList, LinkedList, LinkedHashSet, LinkedHashMap
 - **Non importa**: HashSet o HashMap

Alcune domande utili (2)

- Per una collezione (**non di tipo Map**) , quali operazioni devono essere veloci?
 - Ricerca veloce: HashSet
 - Aggiunta e rimozione di elementi: LinkedList
 - Non importa/ho pochi elementi nella collezione: ArrayList
- Se usate rappresentazioni **ad albero** (TreeSet o TreeMap), serve che si implementi **Comparable**?
 - No, se l'ordine naturale è già prestabilito (es. tipi primitivi)
 - Sì, è un tipo nuovo per il quale vogliamo stabilire l'ordinamento

Altre strutture dati?

- Ci sono due importanti aggiunte al Java Collections Framework
- **Guava**: libreria sviluppata principalmente da Google
- **Apache collections**: libreria del framework Apache





Esercizio

Creare una classe **ElencoDiRoutine** i cui oggetti sono costruiti con una lista di funzioni da stringa a intero da eseguire in sequenza una dopo l'altra

- Queste funzioni sono implementazioni dell'interfaccia `java.util.function.Function`
- La classe dispone inoltre del metodo **esegui** che prende in input un parametro di tipo stringa ed esegue le funzioni in sequenza stampandone l'output
- Costruire quindi un'istanza della classe con le seguenti funzioni (usando riferimenti a metodi laddove possibile):
 1. data in input una stringa `x`, restituisce la lunghezza di `x`
 2. data una stringa `x`, restituisce il numero di occorrenze del carattere `'y'`
 3. data una stringa `x`, restituisce il corrispondente intero (assumendo che `x` contenga solo cifre)
 4. data una stringa `x`, restituisce la somma dei caratteri contenuti nella stringa

Credits

Le slide di questo corso sono il frutto di una personale rielaborazione delle slide del Prof. Navigli.

In aggiunta, le slide sono state revisionate dagli studenti borsisti della Facoltà di Ingegneria Informatica, Informatica e Statistica: Mario Marra e Paolo Straniero.

Font ad alta leggibilità Biancoenero® di biancoenero edizioni srl, disegnata da Umberto Mischi, con la consulenza di Alessandra Finzi, Daniele Zanoni e Luciano Perondi (Brevetto n. RM20110000128).

Disponibile gratuitamente per tutte le istituzioni e i privati che la utilizzino per scopi non commerciali.